enaupuyir

April 6, 2025

0.1 Importar librerias

[1]:
```
import tensorflow as tf
from tensorflow import keras
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import os
import cv2
import json
```

0.2 Descarga y preparación del dataset

[2]:
```
# Cargar credenciales desde 'kaggle.json'
with open('./kaggle.json') as f:
    kaggle_credentials = json.load(f)

os.environ['KAGGLE_USERNAME'] = kaggle_credentials['username']
os.environ['KAGGLE_KEY'] = kaggle_credentials['key']

# Autenticación y descarga del dataset
from kaggle.api.kaggle_api_extended import KaggleApi
api = KaggleApi()
api.authenticate()
api.dataset_download_files('pmigdal/alien-vs-predator-images', path='data',␣
↪unzip=True)
```

```
Dataset URL: https://www.kaggle.com/datasets/pmigdal/alien-vs-predator-images
```

0.3 Exploración de imágenes

[ ]:
```
# Directorio de imágenes y categorías
data_dir = 'data/alien_vs_predator_thumbnails/data/train/'
categories = os.listdir(data_dir)

print("Categorías de seres:", categories)

# Visualización de imágenes
```

```
plt.figure(figsize=(15, 6))
for i, category in enumerate(categories):
    category_dir = os.path.join(data_dir, category)
    images = os.listdir(category_dir)
    plt.subplot(2, 5, i + 1)
    plt.imshow(cv2.imread(os.path.join(category_dir, images[0])))
    plt.title(category)
    plt.axis('off')

plt.tight_layout()
plt.show()
```

```
Categorías de seres: ['alien', 'predator']
```

0.4 Preparación de Datos y División en Entrenamiento, Validación y Test

Entrenamiento = 70%

Validación = 20%

Test = 10%

[26]:
```
data_dir = 'data/alien_vs_predator_thumbnails/data/train/' # Imágenes dentro de␣
↪train

# Recopilación de rutas y etiquetas
filepaths = [os.path.join(dp, f) for dp, dn, filenames in os.walk(data_dir)
```

```
            for f in filenames if os.path.splitext(f)[1].lower() in ['.png','.
 ↪jpg','.jpeg']]
labels = [os.path.split(os.path.split(path)[0])[1] for path in filepaths]

data_dir2 = 'data/alien_vs_predator_thumbnails/data/validation/' # Imágenes␣
 ↪dentro de validation
# Recopilación de rutas y etiquetas
filepaths2 = [os.path.join(dp, f) for dp, dn, filenames in os.walk(data_dir2)
            for f in filenames if os.path.splitext(f)[1].lower() in ['.png','.
 ↪jpg','.jpeg']]

labels2 = [os.path.split(os.path.split(path)[0])[1] for path in filepaths2]

# Unir listas de rutas y etiquetas
filepaths.extend(filepaths2)
labels.extend(labels2)


# Crear un DataFrame con rutas y etiquetas
data_df = pd.DataFrame({'filepath': filepaths, 'label': labels})


# División de datos: 70% entrenamiento, 30% para validación y test (10% test,␣
 ↪20% validación)
from sklearn.model_selection import train_test_split
train_df, test_val_df = train_test_split(data_df, test_size=0.3,␣
 ↪random_state=42)
test_df, val_df = train_test_split(test_val_df, test_size=2/3, random_state=42)

print("Total de imágenes:", len(data_df))
print("Entrenamiento:", len(train_df))
print("Validación:", len(val_df))
print("Test:", len(test_df))
```

```
Total de imágenes: 894
Entrenamiento: 625
Validación: 180
Test: 89
```

0.5 Procesamiento y Generadores de Imágenes

[27]:
```
# Generador para el conjunto de entrenamiento con data augmentation
train_datagen = keras.preprocessing.image.ImageDataGenerator(
    rescale=1./255,
    rotation_range=20,
    horizontal_flip=True
)
```

```
# Generador para validación y test (solo reescala)
test_val_datagen = keras.preprocessing.image.ImageDataGenerator(
    rescale=1./255
)

# Crear los generadores a partir de los DataFrames
train_gen = train_datagen.flow_from_dataframe(
    train_df, x_col='filepath', y_col='label',
    target_size=(100, 100), batch_size=32
)
val_gen = test_val_datagen.flow_from_dataframe(
    val_df, x_col='filepath', y_col='label',
    target_size=(100, 100), batch_size=32, shuffle=False
)
test_gen = test_val_datagen.flow_from_dataframe(
    test_df, x_col='filepath', y_col='label',
    target_size=(100, 100), batch_size=32, shuffle=False
)
```

```
Found 625 validated image filenames belonging to 2 classes.
Found 180 validated image filenames belonging to 2 classes.
Found 89 validated image filenames belonging to 2 classes.
```

0.6 Construcción y Compilación del Modelo

[37]:
```
from keras.models import Sequential
from keras.layers import Dense, Flatten, Dropout

model = Sequential([
    Flatten(input_shape=(100, 100, 3)),
    Dense(512, activation='relu'),
    Dropout(0.2),
    Dense(256, activation='relu'),
    Dense(len(categories), activation='softmax')
])

# Resumen del modelo y su arquitectura
model.summary()

# Compilar el modelo
model.compile(optimizer='adam', # Determina cómo se actualizan los pesos de la␣
 ↪red durante el entrenamiento
              loss='categorical_crossentropy', # Mide qué tan bien el modelo␣
 ↪está haciendo predicciones
              metrics=['accuracy']) # Monitorear el rendimiento del modelo␣
 ↪durante el entrenamiento y la prueba
```

```
Model: "sequential_8"
```

Layer (type)	Output Shape	Param #
flatten_9 (Flatten)	(None, 30000)	0
dense_26 (Dense)	(None, 512)	15,360,512
dropout_9 (Dropout)	(None, 512)	0
dense_27 (Dense)	(None, 256)	131,328
dense_28 (Dense)	(None, 2)	514

```
Total params: 15,492,354 (59.10 MB)

Trainable params: 15,492,354 (59.10 MB)

Non-trainable params: 0 (0.00 B)
```

1 Entrenamiento del Modelo

[38]:
```
from keras.callbacks import EarlyStopping, ReduceLROnPlateau, ModelCheckpoint

model_checkpoint = ModelCheckpoint('best_model.h5', save_best_only=True,
                                   monitor='val_loss', mode='min')
early_stopping = EarlyStopping(monitor='val_loss', patience=10, verbose=1,␣
 ↪mode='min')
reduce_lr = ReduceLROnPlateau(monitor='val_loss', factor=0.1, patience=5,␣
 ↪verbose=1, mode='min')

callbacks = [model_checkpoint, early_stopping, reduce_lr]

with tf.device('/device:GPU:0'):
    history = model.fit(
        train_gen,
        validation_data=val_gen,
        epochs=50,
        callbacks=callbacks
    )
```

```
c:\Users\diego\AppData\Local\Programs\Python\Python312\Lib\site-
```

```
packages\keras\src\trainers\data_adapters\py_dataset_adapter.py:121:
UserWarning: Your `PyDataset` class should call `super().__init__(**kwargs)` in
its constructor. `**kwargs` can include `workers`, `use_multiprocessing`,
`max_queue_size`. Do not pass these arguments to `fit()`, as they will be
ignored.
  self._warn_if_super_not_called()

Epoch 1/50
20/20              0s 433ms/step -
accuracy: 0.5271 - loss: 10.6370

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or
`keras.saving.save_model(model)`. This file format is considered legacy. We
recommend using instead the native Keras format, e.g.
`model.save('my_model.keras')` or `keras.saving.save_model(model,
'my_model.keras')`.

20/20              12s 541ms/step -
accuracy: 0.5270 - loss: 10.4882 - val_accuracy: 0.4778 - val_loss: 1.6195 -
learning_rate: 0.0010
Epoch 2/50
20/20              4s 212ms/step -
accuracy: 0.5563 - loss: 2.0226 - val_accuracy: 0.4944 - val_loss: 2.2551 -
learning_rate: 0.0010
Epoch 3/50
20/20              4s 210ms/step -
accuracy: 0.5565 - loss: 1.2806 - val_accuracy: 0.4722 - val_loss: 2.4869 -
learning_rate: 0.0010
Epoch 4/50
20/20              0s 191ms/step -
accuracy: 0.5601 - loss: 1.2494

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or
`keras.saving.save_model(model)`. This file format is considered legacy. We
recommend using instead the native Keras format, e.g.
`model.save('my_model.keras')` or `keras.saving.save_model(model,
'my_model.keras')`.

20/20              25s 1s/step -
accuracy: 0.5595 - loss: 1.2432 - val_accuracy: 0.6222 - val_loss: 0.7888 -
learning_rate: 0.0010
Epoch 5/50
20/20              4s 206ms/step -
accuracy: 0.5747 - loss: 0.9433 - val_accuracy: 0.5167 - val_loss: 1.2825 -
learning_rate: 0.0010
Epoch 6/50
20/20              4s 206ms/step -
accuracy: 0.5760 - loss: 0.9732 - val_accuracy: 0.5389 - val_loss: 0.9464 -
learning_rate: 0.0010
Epoch 7/50
```

```
20/20              0s 192ms/step -
accuracy: 0.6266 - loss: 0.7743

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or
`keras.saving.save_model(model)`. This file format is considered legacy. We
recommend using instead the native Keras format, e.g.
`model.save('my_model.keras')` or `keras.saving.save_model(model,
'my_model.keras')`.

20/20              4s 224ms/step -
accuracy: 0.6261 - loss: 0.7746 - val_accuracy: 0.5889 - val_loss: 0.6664 -
learning_rate: 0.0010
Epoch 8/50
20/20              0s 193ms/step -
accuracy: 0.5705 - loss: 0.8143

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or
`keras.saving.save_model(model)`. This file format is considered legacy. We
recommend using instead the native Keras format, e.g.
`model.save('my_model.keras')` or `keras.saving.save_model(model,
'my_model.keras')`.

20/20              5s 224ms/step -
accuracy: 0.5705 - loss: 0.8149 - val_accuracy: 0.6222 - val_loss: 0.6639 -
learning_rate: 0.0010
Epoch 9/50
20/20              4s 209ms/step -
accuracy: 0.5684 - loss: 0.7962 - val_accuracy: 0.5278 - val_loss: 0.7325 -
learning_rate: 0.0010
Epoch 10/50
20/20              0s 191ms/step -
accuracy: 0.5293 - loss: 0.7597

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or
`keras.saving.save_model(model)`. This file format is considered legacy. We
recommend using instead the native Keras format, e.g.
`model.save('my_model.keras')` or `keras.saving.save_model(model,
'my_model.keras')`.

20/20              4s 222ms/step -
accuracy: 0.5322 - loss: 0.7577 - val_accuracy: 0.6500 - val_loss: 0.6442 -
learning_rate: 0.0010
Epoch 11/50
20/20              4s 208ms/step -
accuracy: 0.6106 - loss: 0.6555 - val_accuracy: 0.6222 - val_loss: 0.7567 -
learning_rate: 0.0010
Epoch 12/50
20/20              4s 209ms/step -
accuracy: 0.6219 - loss: 0.7057 - val_accuracy: 0.6444 - val_loss: 0.6488 -
learning_rate: 0.0010
Epoch 13/50
```

```
20/20              4s 207ms/step -
accuracy: 0.6119 - loss: 0.6396 - val_accuracy: 0.6667 - val_loss: 0.6771 -
learning_rate: 0.0010
Epoch 14/50
20/20              4s 206ms/step -
accuracy: 0.6416 - loss: 0.6686 - val_accuracy: 0.6556 - val_loss: 0.6512 -
learning_rate: 0.0010
Epoch 15/50
20/20              0s 194ms/step -
accuracy: 0.6260 - loss: 0.6696
Epoch 15: ReduceLROnPlateau reducing learning rate to 0.00010000000474974513.
20/20              4s 205ms/step -
accuracy: 0.6266 - loss: 0.6689 - val_accuracy: 0.6667 - val_loss: 0.6907 -
learning_rate: 0.0010
Epoch 16/50
20/20              4s 204ms/step -
accuracy: 0.6080 - loss: 0.6980 - val_accuracy: 0.6333 - val_loss: 0.6469 -
learning_rate: 1.0000e-04
Epoch 17/50
20/20              0s 194ms/step -
accuracy: 0.6849 - loss: 0.6118

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or
`keras.saving.save_model(model)`. This file format is considered legacy. We
recommend using instead the native Keras format, e.g.
`model.save('my_model.keras')` or `keras.saving.save_model(model,
'my_model.keras')`.

20/20              10s 505ms/step -
accuracy: 0.6840 - loss: 0.6120 - val_accuracy: 0.6333 - val_loss: 0.6425 -
learning_rate: 1.0000e-04
Epoch 18/50
20/20              0s 195ms/step -
accuracy: 0.6822 - loss: 0.6297

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or
`keras.saving.save_model(model)`. This file format is considered legacy. We
recommend using instead the native Keras format, e.g.
`model.save('my_model.keras')` or `keras.saving.save_model(model,
'my_model.keras')`.

20/20              4s 226ms/step -
accuracy: 0.6817 - loss: 0.6286 - val_accuracy: 0.6222 - val_loss: 0.6420 -
learning_rate: 1.0000e-04
Epoch 19/50
20/20              0s 194ms/step -
accuracy: 0.6651 - loss: 0.6054

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or
`keras.saving.save_model(model)`. This file format is considered legacy. We
```

```
recommend using instead the native Keras format, e.g.
`model.save('my_model.keras')` or `keras.saving.save_model(model,
'my_model.keras')`.

20/20              4s 223ms/step -
accuracy: 0.6655 - loss: 0.6055 - val_accuracy: 0.6389 - val_loss: 0.6381 -
learning_rate: 1.0000e-04
Epoch 20/50
20/20              4s 210ms/step -
accuracy: 0.6650 - loss: 0.5959 - val_accuracy: 0.6444 - val_loss: 0.6404 -
learning_rate: 1.0000e-04
Epoch 21/50
20/20              4s 204ms/step -
accuracy: 0.6938 - loss: 0.5831 - val_accuracy: 0.6444 - val_loss: 0.6433 -
learning_rate: 1.0000e-04
Epoch 22/50
20/20              4s 203ms/step -
accuracy: 0.6507 - loss: 0.6235 - val_accuracy: 0.6333 - val_loss: 0.6395 -
learning_rate: 1.0000e-04
Epoch 23/50
20/20              0s 190ms/step -
accuracy: 0.6788 - loss: 0.5942

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or
`keras.saving.save_model(model)`. This file format is considered legacy. We
recommend using instead the native Keras format, e.g.
`model.save('my_model.keras')` or `keras.saving.save_model(model,
'my_model.keras')`.

20/20              4s 223ms/step -
accuracy: 0.6785 - loss: 0.5943 - val_accuracy: 0.6333 - val_loss: 0.6373 -
learning_rate: 1.0000e-04
Epoch 24/50
20/20              0s 194ms/step -
accuracy: 0.6997 - loss: 0.5586

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or
`keras.saving.save_model(model)`. This file format is considered legacy. We
recommend using instead the native Keras format, e.g.
`model.save('my_model.keras')` or `keras.saving.save_model(model,
'my_model.keras')`.

20/20              5s 230ms/step -
accuracy: 0.6988 - loss: 0.5597 - val_accuracy: 0.6389 - val_loss: 0.6347 -
learning_rate: 1.0000e-04
Epoch 25/50
20/20              0s 194ms/step -
accuracy: 0.5973 - loss: 0.6281

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or
`keras.saving.save_model(model)`. This file format is considered legacy. We
```

```
recommend using instead the native Keras format, e.g.
`model.save('my_model.keras')` or `keras.saving.save_model(model,
'my_model.keras')`.

20/20              6s 280ms/step -
accuracy: 0.5995 - loss: 0.6272 - val_accuracy: 0.6500 - val_loss: 0.6331 -
learning_rate: 1.0000e-04
Epoch 26/50
20/20              0s 196ms/step -
accuracy: 0.6472 - loss: 0.5966

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or
`keras.saving.save_model(model)`. This file format is considered legacy. We
recommend using instead the native Keras format, e.g.
`model.save('my_model.keras')` or `keras.saving.save_model(model,
'my_model.keras')`.

20/20              7s 335ms/step -
accuracy: 0.6482 - loss: 0.5964 - val_accuracy: 0.6333 - val_loss: 0.6293 -
learning_rate: 1.0000e-04
Epoch 27/50
20/20              4s 210ms/step -
accuracy: 0.6695 - loss: 0.5718 - val_accuracy: 0.6389 - val_loss: 0.6326 -
learning_rate: 1.0000e-04
Epoch 28/50
20/20              0s 196ms/step -
accuracy: 0.7123 - loss: 0.5686

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or
`keras.saving.save_model(model)`. This file format is considered legacy. We
recommend using instead the native Keras format, e.g.
`model.save('my_model.keras')` or `keras.saving.save_model(model,
'my_model.keras')`.

20/20              7s 334ms/step -
accuracy: 0.7114 - loss: 0.5696 - val_accuracy: 0.6333 - val_loss: 0.6291 -
learning_rate: 1.0000e-04
Epoch 29/50
20/20              4s 203ms/step -
accuracy: 0.6751 - loss: 0.5890 - val_accuracy: 0.6444 - val_loss: 0.6308 -
learning_rate: 1.0000e-04
Epoch 30/50
20/20              4s 201ms/step -
accuracy: 0.6623 - loss: 0.5834 - val_accuracy: 0.6556 - val_loss: 0.6342 -
learning_rate: 1.0000e-04
Epoch 31/50
20/20              4s 203ms/step -
accuracy: 0.7174 - loss: 0.5457 - val_accuracy: 0.6556 - val_loss: 0.6302 -
learning_rate: 1.0000e-04
Epoch 32/50
```

```
20/20              4s 209ms/step -
accuracy: 0.6657 - loss: 0.6023 - val_accuracy: 0.6444 - val_loss: 0.6307 -
learning_rate: 1.0000e-04
Epoch 33/50
20/20              0s 195ms/step -
accuracy: 0.6556 - loss: 0.5678
Epoch 33: ReduceLROnPlateau reducing learning rate to 1.0000000474974514e-05.
20/20              4s 208ms/step -
accuracy: 0.6568 - loss: 0.5677 - val_accuracy: 0.6500 - val_loss: 0.6337 -
learning_rate: 1.0000e-04
Epoch 34/50
20/20              4s 208ms/step -
accuracy: 0.6965 - loss: 0.5891 - val_accuracy: 0.6444 - val_loss: 0.6316 -
learning_rate: 1.0000e-05
Epoch 35/50
20/20              4s 204ms/step -
accuracy: 0.6609 - loss: 0.5673 - val_accuracy: 0.6444 - val_loss: 0.6304 -
learning_rate: 1.0000e-05
Epoch 36/50
20/20              4s 202ms/step -
accuracy: 0.7158 - loss: 0.5503 - val_accuracy: 0.6611 - val_loss: 0.6292 -
learning_rate: 1.0000e-05
Epoch 37/50
20/20              0s 190ms/step -
accuracy: 0.7051 - loss: 0.5789

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or
`keras.saving.save_model(model)`. This file format is considered legacy. We
recommend using instead the native Keras format, e.g.
`model.save('my_model.keras')` or `keras.saving.save_model(model,
'my_model.keras')`.

20/20              6s 331ms/step -
accuracy: 0.7042 - loss: 0.5787 - val_accuracy: 0.6444 - val_loss: 0.6286 -
learning_rate: 1.0000e-05
Epoch 38/50
20/20              0s 191ms/step -
accuracy: 0.7047 - loss: 0.5501

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or
`keras.saving.save_model(model)`. This file format is considered legacy. We
recommend using instead the native Keras format, e.g.
`model.save('my_model.keras')` or `keras.saving.save_model(model,
'my_model.keras')`.

20/20              7s 333ms/step -
accuracy: 0.7045 - loss: 0.5507 - val_accuracy: 0.6444 - val_loss: 0.6279 -
learning_rate: 1.0000e-05
Epoch 39/50
20/20              0s 192ms/step -
```

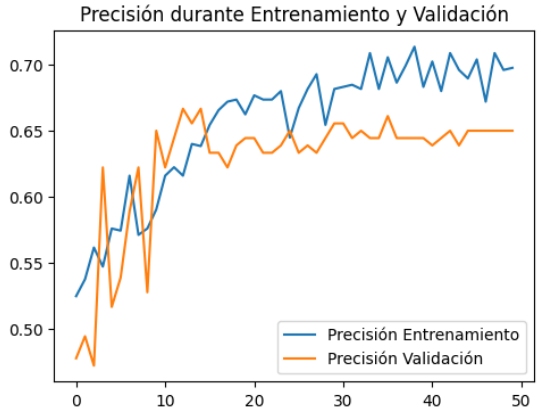
```
accuracy: 0.7456 - loss: 0.5262

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or
`keras.saving.save_model(model)`. This file format is considered legacy. We
recommend using instead the native Keras format, e.g.
`model.save('my_model.keras')` or `keras.saving.save_model(model,
'my_model.keras')`.

20/20            6s 329ms/step -
accuracy: 0.7441 - loss: 0.5270 - val_accuracy: 0.6444 - val_loss: 0.6276 -
learning_rate: 1.0000e-05
Epoch 40/50
20/20            4s 204ms/step -
accuracy: 0.6490 - loss: 0.5664 - val_accuracy: 0.6444 - val_loss: 0.6281 -
learning_rate: 1.0000e-05
Epoch 41/50
20/20            4s 204ms/step -
accuracy: 0.7145 - loss: 0.5585 - val_accuracy: 0.6389 - val_loss: 0.6278 -
learning_rate: 1.0000e-05
Epoch 42/50
20/20            0s 192ms/step -
accuracy: 0.7012 - loss: 0.5471

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or
`keras.saving.save_model(model)`. This file format is considered legacy. We
recommend using instead the native Keras format, e.g.
`model.save('my_model.keras')` or `keras.saving.save_model(model,
'my_model.keras')`.

20/20            6s 326ms/step -
accuracy: 0.7002 - loss: 0.5481 - val_accuracy: 0.6444 - val_loss: 0.6274 -
learning_rate: 1.0000e-05
Epoch 43/50
20/20            0s 194ms/step -
accuracy: 0.7311 - loss: 0.5479

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or
`keras.saving.save_model(model)`. This file format is considered legacy. We
recommend using instead the native Keras format, e.g.
`model.save('my_model.keras')` or `keras.saving.save_model(model,
'my_model.keras')`.

20/20            7s 344ms/step -
accuracy: 0.7300 - loss: 0.5488 - val_accuracy: 0.6500 - val_loss: 0.6268 -
learning_rate: 1.0000e-05
Epoch 44/50
20/20            4s 202ms/step -
accuracy: 0.6907 - loss: 0.5566 - val_accuracy: 0.6389 - val_loss: 0.6271 -
learning_rate: 1.0000e-05
Epoch 45/50
20/20            4s 205ms/step -
```

```
accuracy: 0.6705 - loss: 0.5953 - val_accuracy: 0.6500 - val_loss: 0.6274 -
learning_rate: 1.0000e-05
Epoch 46/50
20/20              4s 202ms/step -
accuracy: 0.6847 - loss: 0.5797 - val_accuracy: 0.6500 - val_loss: 0.6278 -
learning_rate: 1.0000e-05
Epoch 47/50
20/20              4s 205ms/step -
accuracy: 0.6820 - loss: 0.5476 - val_accuracy: 0.6500 - val_loss: 0.6279 -
learning_rate: 1.0000e-05
Epoch 48/50
20/20              0s 195ms/step -
accuracy: 0.7155 - loss: 0.5547
Epoch 48: ReduceLROnPlateau reducing learning rate to 1.0000000656873453e-06.
20/20              4s 205ms/step -
accuracy: 0.7152 - loss: 0.5547 - val_accuracy: 0.6500 - val_loss: 0.6270 -
learning_rate: 1.0000e-05
Epoch 49/50
20/20              4s 207ms/step -
accuracy: 0.6765 - loss: 0.5767 - val_accuracy: 0.6500 - val_loss: 0.6270 -
learning_rate: 1.0000e-06
Epoch 50/50
20/20              4s 207ms/step -
accuracy: 0.6912 - loss: 0.5621 - val_accuracy: 0.6500 - val_loss: 0.6270 -
learning_rate: 1.0000e-06
```

1.1 Evaluación modelo

[39]:
```python
# Visualización de la evolución del entrenamiento
acc = history.history['accuracy']
val_acc = history.history['val_accuracy']
loss = history.history['loss']
val_loss = history.history['val_loss']

epochs_range = range(len(acc))

plt.figure(figsize=(12, 4))
plt.subplot(1, 2, 1)
plt.plot(epochs_range, acc, label='Precisión Entrenamiento')
plt.plot(epochs_range, val_acc, label='Precisión Validación')
plt.legend(loc='lower right')
plt.title('Precisión durante Entrenamiento y Validación')

plt.subplot(1, 2, 2)
plt.plot(epochs_range, loss, label='Pérdida Entrenamiento')
plt.plot(epochs_range, val_loss, label='Pérdida Validación')
plt.legend(loc='upper right')
```

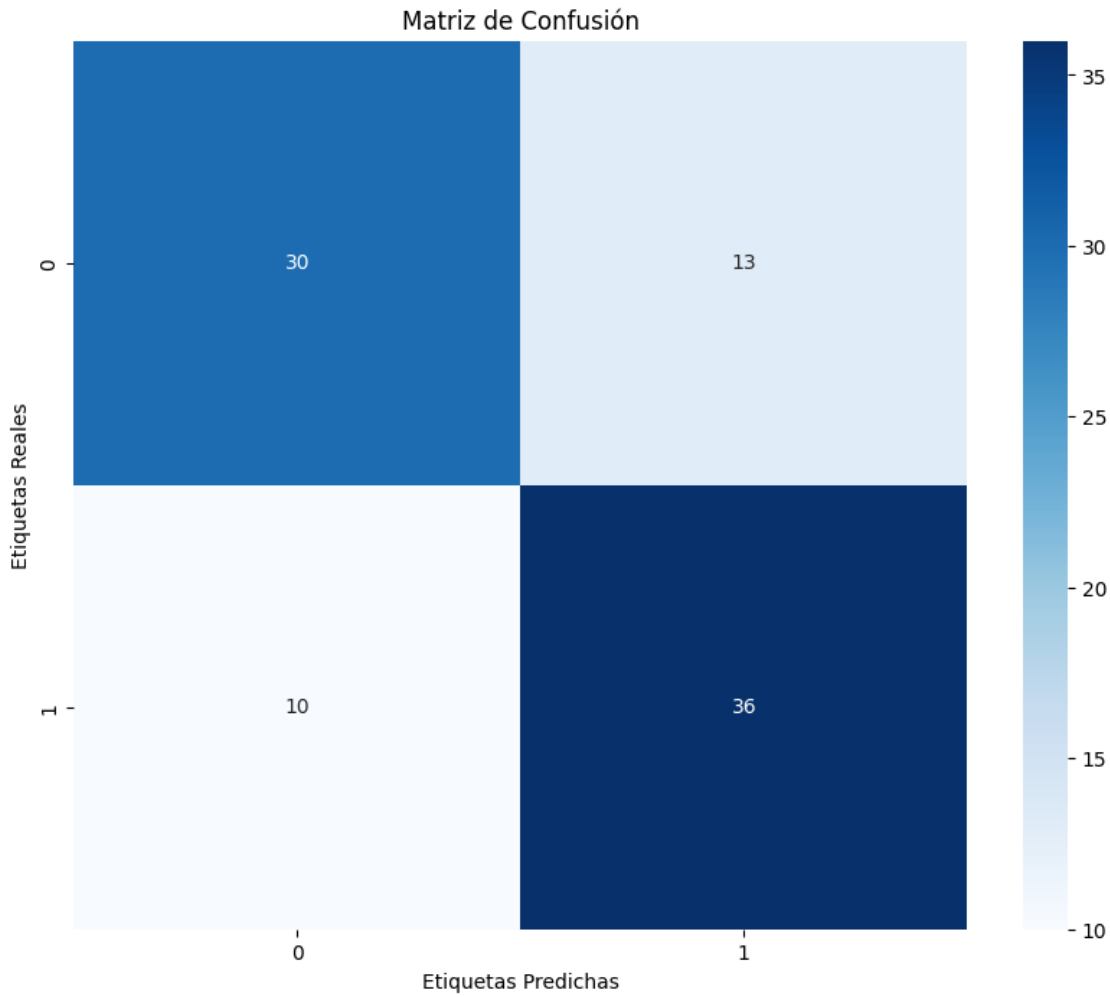
```python
plt.title('Pérdida durante Entrenamiento y Validación')
plt.show()

from keras.models import load_model

# Cargar el mejor modelo guardado
best_model = load_model('best_model.h5')

# Evaluación en el conjunto de test
import numpy as np
from sklearn.metrics import confusion_matrix, classification_report
import seaborn as sns

test_gen.reset()
predictions = best_model.predict(test_gen, verbose=1)
y_pred = np.argmax(predictions, axis=1)
y_test = test_gen.classes

# Matriz de Confusión
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(10, 8))
sns.heatmap(cm, annot=True, fmt='d', cmap=plt.cm.Blues)
plt.title('Matriz de Confusión')
plt.ylabel('Etiquetas Reales')
plt.xlabel('Etiquetas Predichas')
plt.show()

# Reporte de Clasificación
print(classification_report(y_test, y_pred, target_names=categories))
```



```
WARNING:absl:Compiled the loaded model, but the compiled metrics have yet to be
built. `model.compile_metrics` will be empty until you train or evaluate the
model.
```

```
c:\Users\diego\AppData\Local\Programs\Python\Python312\Lib\site-
packages\keras\src\trainers\data_adapters\py_dataset_adapter.py:121:
UserWarning: Your `PyDataset` class should call `super().__init__(**kwargs)` in
its constructor. `**kwargs` can include `workers`, `use_multiprocessing`,
`max_queue_size`. Do not pass these arguments to `fit()`, as they will be
ignored.
  self._warn_if_super_not_called()
```

```
3/3              0s 135ms/step
```




```
              precision    recall  f1-score   support

       alien       0.75      0.70      0.72        43
    predator       0.73      0.78      0.76        46

    accuracy                           0.74        89
   macro avg       0.74      0.74      0.74        89
```

```
weighted avg       0.74      0.74      0.74        89
```